

Article

Research on the A* Algorithm Based on Adaptive Weights and Heuristic Reward Values

Xizheng Wang, Gang Li *  and Zijian Bian

School of Automobile and Traffic Engineering, Liaoning University of Technology, Jinzhou 121001, China; 231285063@stu.lnut.edu.cn (X.W.); bianzijian295@163.com (Z.B.)

* Correspondence: qcxyiligang@lnut.edu.cn

Abstract: Aiming at the problems of the A* algorithm's long running time, large number of search nodes, tortuous paths, and the planned paths being prone to colliding with the corner points of obstacles, adaptive weighting and reward value theory are proposed to improve it. Firstly, the diagonal-free five-way search based on the number of coordinate changes is used to make the algorithm purposeful. Meanwhile, in order to improve the path security, the diagonal search is filtered out when there are obstacles in the search neighborhood. Secondly, a radial basis function is used to act as the adaptive weighting coefficient of the heuristic function and adjust the proportion of heuristic functions in the algorithm accordingly to the search distance. Again, optimize the cost function using the reward value provided by the target point so that the current point is away from the local optimum. Finally, a secondary optimization of the path is performed to increase the distance between the path and the barriers, and the optimized path is smoothed using Bessel curves. Typical working conditions are selected, and the algorithm is verified through simulation tests. Simulation tests show that the algorithm not only shortens the planning time and improves the path security but also reduces the number of search nodes by about 76.4% on average and the turn angle by about 71.7% on average.

Keywords: path planning; dynamic weights; secondary optimization of paths; Bessel curves



Academic Editor: Aymeric Rousseau

Received: 17 January 2025

Revised: 18 February 2025

Accepted: 1 March 2025

Published: 4 March 2025

Citation: Wang, X.; Li, G.; Bian, Z. Research on the A* Algorithm Based on Adaptive Weights and Heuristic Reward Values. *World Electr. Veh. J.* **2025**, *16*, 144. <https://doi.org/10.3390/wevj16030144>

Copyright: © 2025 by the authors. Published by MDPI on behalf of the World Electric Vehicle Association. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The swift progression of autonomous driving technology has made path planning for self-driving cars increasingly crucial. It is highly significant to enhance the quality of human life and traffic safety. As the core of intelligent driving, path planning plays an important role, so its algorithms have been the focus of research by a multitude of scholars in recent years. Dijkstra's algorithm [1], A* algorithm [2], RRT algorithm [3], and ant colony algorithm [4] have been widely studied by scholars. While the A* algorithm is capable of planning the optimal path, the path planned by the algorithm is more winding and has a lot of redundant points [5]. Erke S et al. [6] proposed the A* algorithm, which states that the search step can be changed; for the traditional algorithm of high time and space complexity, it dynamically adjusts the search step according to the position of the obstacle. In the open area, large-step search, close to the obstacle, and small-step search both reduce the computation time but also ensure the effectiveness of path planning. Gang T et al. [7] eliminate redundant points by constructing functions to achieve the effect of simplifying the path and avoiding the appearance of jagged paths. Dan X et al. [8], a multi-objective path-planning strategy based on a greedy algorithm and improved A* algorithm, is proposed. Jing Z et al. [9] improved the robustness of the algorithm in complex environments by

incorporating the repulsive and attractive forces of the APF method into the search direction of the heuristic function. Xiaoxiao L et al. [10] proposed the method of extending the search neighborhood by increasing the search directions from eight to 16 by adding more search layers. Lisang L et al. [11] proposed a new search rule: when the algorithm encounters an obstacle, the node with the lowest cost among the neighboring points is used as the parent node for the next loop. This rule reduces the algorithm's time consumption and reduces the amount of computation. Huixia Z et al. [12] and Xiangrong T et al. [13]: the initial and final points are connected by the vertical distance, which is utilized as the deviation coefficient to optimize the heuristic function. This process reduces the number of search nodes and the time required for the search, making it less time-consuming, reduces the unnecessary search area, and improves the search efficiency. Mengxue Y et al. [14] proposed a weighted average distance formula, which adjusts the ratio of the diagonal distance to the Manhattan distance through the balancing coefficient and adapts to different map environments. This process is capable of balancing the search speed with the accuracy of the optimal path. In order for the algorithm to take into account the location of the target point when planning the path, the APF is combined with A*, and the search direction is determined by the direction of the force [15]. Peiying L et al. [16] and Chuanwei Z et al. [17] improved the evaluation function by introducing the obstacle rate to make the algorithm more adaptive and globally optimized. Xingdong W et al. [18] improved the evaluation function so that the algorithm's planning paths take into account road conditions. Yangqi O et al. [19] proposed a smooth path strategy and path node mechanism to make the robot easy to track. Zhi R et al. [20] introduced a shrinkage factor for the A* algorithm and optimized the shrinkage factor through iterative training with reinforcement learning to effectively improve the algorithm's time performance without sacrificing the optimality of too many planning results. Kvitko D et al. [21] proposed to design a path-planning algorithm based on the Courbage–Nekorkin artificial neuron model with overlay control parameters based on chaotic behaviors, which solves the problems of blind search and low efficiency of traditional path-planning algorithms.

Challenges faced by A* algorithms in path planning mainly include too many search nodes, a long running time, and too many corners of the planned path, which are reflected in the field of unmanned and robotic path planning. When in a complex environment, the A* algorithm needs to evaluate more nodes to find the optimal path, resulting in high computational resource consumption and a longer response time, especially when there are complex obstacles. Obstacles will constantly block the expansion of the current node, and the A* algorithm needs to evaluate a large number of nodes, and the computation time increases dramatically, which affects the real-time navigation. Therefore, improving the search strategy and heuristic function becomes the key to improving the performance.

Based on the common problems of A* algorithms in the above studies, the following work is carried out in this paper to address the drawbacks of long running time, many search nodes, collision of planned paths with the corners of obstacles, and many redundant points of paths:

- (1) The traditional eight-neighborhood search is optimized to a diagonal-free five-way search based on coordinate difference, which filters out the diagonal search when there is an obstacle near the neighborhood and improves the path security;
- (2) Add adaptive dynamic weighting coefficients for the heuristic function when the current point is nearer to the starting point, effectively reducing the quantity of search nodes to a minimum. As the current point gets closer to the goal, the algorithm explores more nodes to find a path that is relatively optimal;

(3) Add the heuristic reward, so that when there is a phenomenon of localized optimality, the use of the heuristic reward value will help the current point escape from localized optimality and also decrease the quantity of search nodes and the time consumption;

(4) A secondary optimisation of the path is carried out, and the optimised path not only removes redundant points, but also further increases the distance to the obstacles;

(5) Path smoothing is achieved by adding auxiliary points to the Bessel curve. Finally, raster maps for three sets of operating conditions are plotted to verify the effectiveness of the improved algorithm presented in this paper.

2. Improvement of the A* Algorithm

In this paper, the improvement will be carried out in the following six parts: Section 2.1 introduces the principles of the A* algorithm, compares traditional search methods and heuristic function formulations, and optimizes and improves on eight-neighbor search and Euclidean distance; Section 2.2 improves the expansion of the child nodes and also proposes a coordinate difference-based diagonal-free five-way search in order to improve the quality of the paths; Section 2.3 In order to satisfy different search strategies in different weighting cases, adaptive dynamic weighting coefficients are added to the heuristic function; Section 2.4 In order to ensure the quality of the paths and at the same time reduce the algorithm's time consumption and the number of searching nodes to a greater extent, a reward value is added to the cost function; Section 2.5 performs a secondary optimization of the paths to improve the quality of the paths while eliminating redundant path points; Section 2.6 compares Bessel curves with cubic B-spline curves, and finally, Bessel curves are chosen to smooth the paths planned by the improved A* algorithm.

2.1. A* Algorithm Principles

The A* algorithm adds heuristics to the Dijkstra algorithm [22]. The A* algorithm considers not only the distance cost associated with the current node but also the expected future cost and typically employs an eight-neighborhood search and a four-neighborhood search. The eight-neighborhood search algorithm inspects nodes located in the eight surrounding directions of the current node, allowing for diagonal movement and allowing exploration of more of the search space. Four-neighborhood search focuses on the four cardinal directions of the current node, providing a more streamlined and direct approach to pathfinding, which is particularly beneficial in certain grid-based environments. The A* algorithm also exists in a higher neighborhood search, but compared to the commonly used search neighborhood, it may increase the amount of computation, so it is not considered in this paper, and only the four-neighborhood and eight-neighborhood are chosen.

The eight-neighborhood search and four-neighborhood search are compared; pink is the eight-neighborhood search nodes, and green is the four-neighborhood compared to the eight-neighborhood more searched nodes. The search is shown in Figure 1. The path length, number of corners, algorithm time, and number of search nodes are shown in Table 1.

It is evident that the paths planned by four-neighborhood search, when compared with those planned by eight-neighborhood search, exhibit a greater number of corners, longer path lengths, and more search nodes. Therefore, this paper innovates on the basis of eight-neighborhood search.

The evaluation function for the A* algorithm is shown below:

$$f(n) = g(n) + h(n) \quad (1)$$

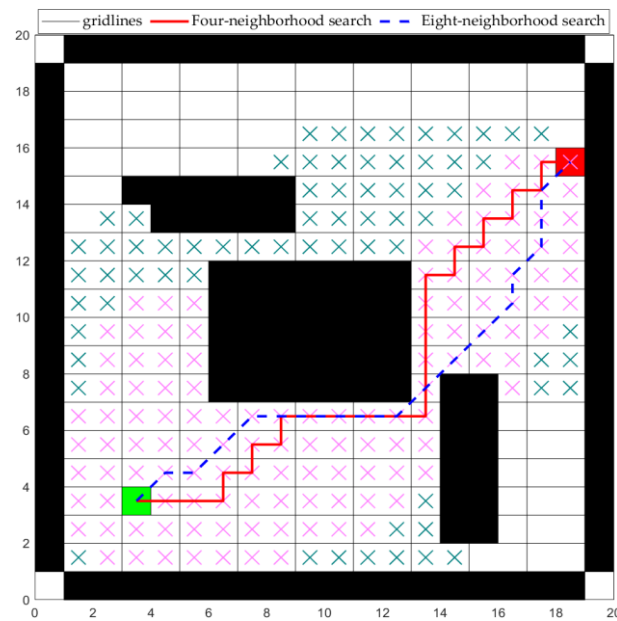


Figure 1. Comparison of eight-neighborhood and four-neighborhood searches.

Table 1. Search results for eight-neighborhood and four-neighborhood.

Search Method	Algorithm Time/s	Number of Search Nodes	Number of Corners	Path Length/m
Eight-neighborhood	1.31	125	8	21.72
Four-neighborhood	1.30	192	16	27.00

In this study, the variable n is used to denote the current node, while $f(n)$ is used to denote the total consumption cost. The variable $g(n)$ is used to denote the actual consumption cost from the current node to the starting point, and $h(n)$ is used to denote the estimated cost from the current point to the target point.

The determination of the heuristic function $h(n)$ is typically performed using three distinct distances: the Manhattan distance, the Euclidean distance, and the Chebyshev distance. Then Chebyshev distance cannot directly reflect the distance between two points [23], so in this paper we only compare Manhattan distance and Euclidean distance and use one of them.

The formula for calculating the Euclidean distance is similar to that for calculating the distance from the hypotenuse of a triangle, which is shown below:

$$d_1 = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (2)$$

The Manhattan distance is calculated based on the difference in coordinates between two points. The formula for this calculation is shown below:

$$d_2 = |x_1 - x_2| + |y_1 - y_2| \quad (3)$$

Comparing the Euclidean distance to the Manhattan distance, the comparison is schematically shown in Figure 2.

The red arrows indicate the Euclidean distance, and the blue arrows indicate the Manhattan distance, which, as can be seen from the figure, is slightly longer than the actual length. Therefore, the Euclidean distance was used in this study.

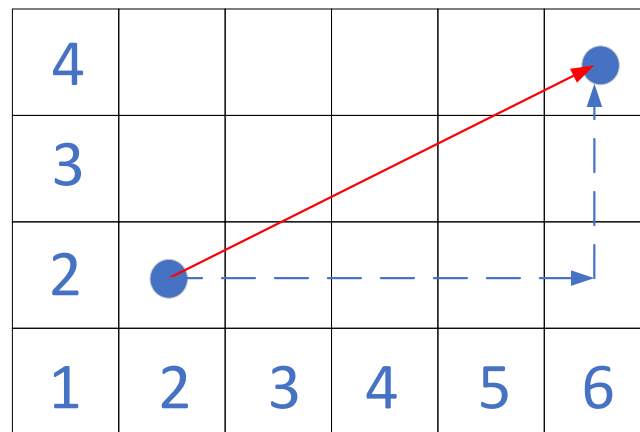


Figure 2. Comparison chart of Manhattan distance and Euclidean distance.

2.2. Optimize the Way Child Nodes Expand

A* the key to this algorithm is to maintain an open list through an iterative process. In each iteration, the node with the smallest total consumption cost in the open list is designated as the parent node, and then child nodes are expanded from the parent. The new child nodes have nodes in the opposite direction of the goal point, and expanding these nodes wastes time, and the search direction of the current node is shown in Figure 3:

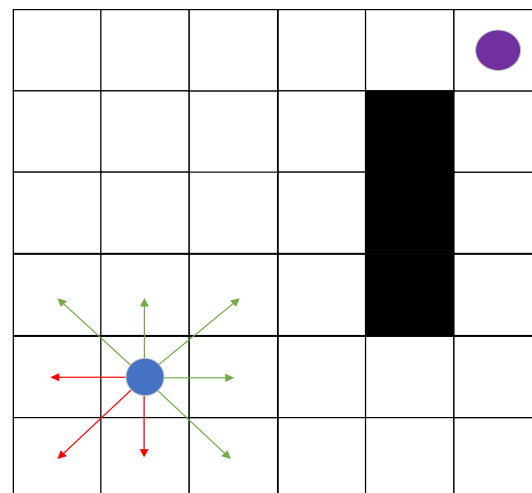


Figure 3. Current node search direction graph.

As illustrated in Figure 3, the purple color indicates the target node, while the blue color denotes the starting point. The figure demonstrates that, when the current node is extended, only five green lines are useful; the remaining three red lines are redundant. In this paper, we employ the diagonal-free five-way search based on the number of coordinate changes, which is extended as shown in Figure 4, and the extension formula is shown as follows:

$$\begin{cases} \Delta x = x_n - x_{goal} \\ \Delta y = y_n - y_{goal} \end{cases} \quad (4)$$

where (x_n, y_n) are the current point coordinates, and (x_{goal}, y_{goal}) are the target point coordinates.

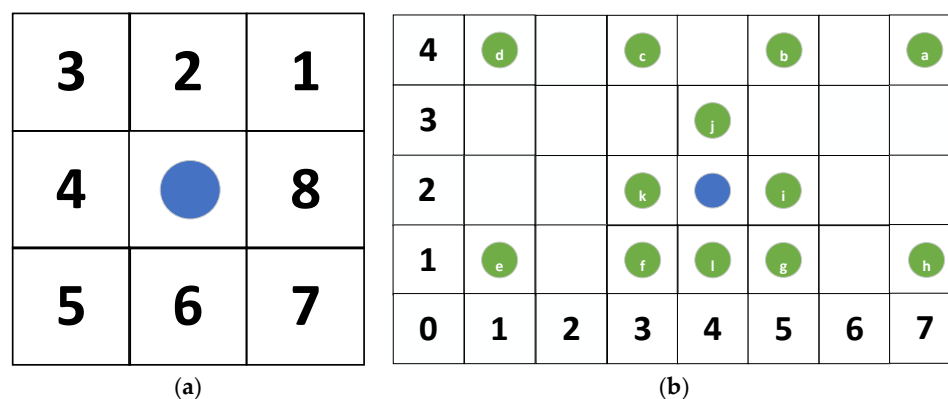


Figure 4. Five-way search based on coordinate differences. They are: (a) Neighborhood around the current node and (b) Relative position of the current node to the target point.

In the figure, the current node is shown in blue, and the target nodes with different orientations are shown in green. The values of Δx and Δy are calculated separately, and the search neighborhood is shown below:

$$\text{search neighborhood} = \begin{cases} \{1, 2, 3, 7, 8\}, \Delta x < 0 \cap \Delta y < 0 \cap |\Delta x| > |\Delta y| \\ \{1, 2, 3, 4, 8\}, \Delta x \neq 0 \cap \Delta y < 0 \cap |\Delta x| \leq |\Delta y| \\ \{4, 5, 6, 7, 8\}, \Delta x \neq 0 \cap \Delta y > 0 \cap |\Delta x| \leq |\Delta y| \\ \{1, 5, 6, 7, 8\}, \Delta x < 0 \cap \Delta y > 0 \cap |\Delta x| > |\Delta y| \\ \{1, 2, 3, 4, 5\}, \Delta x > 0 \cap \Delta y < 0 \cap |\Delta x| > |\Delta y| \\ \{3, 4, 5, 6, 7\}, \Delta x > 0 \cap \Delta y > 0 \cap |\Delta x| > |\Delta y| \\ \{1, 2, 6, 7, 8\}, \Delta x < 0 \cap \Delta y = 0 \\ \{1, 2, 3, 4, 8\}, \Delta x = 0 \cap \Delta y < 0 \\ \{2, 3, 4, 5, 6\}, \Delta x > 0 \cap \Delta y = 0 \\ \{4, 5, 6, 7, 8\}, \Delta x = 0 \cap \Delta y > 0 \end{cases} \quad (5)$$

The five-way search based on coordinate difference is compared with the eight-neighborhood search; the pink color is the node of the five-way search based on coordinate difference, and the green color is the node of the eight-neighborhood multi-search. The comparison graph is shown in Figure 5, and the comparison results are shown in Table 2.

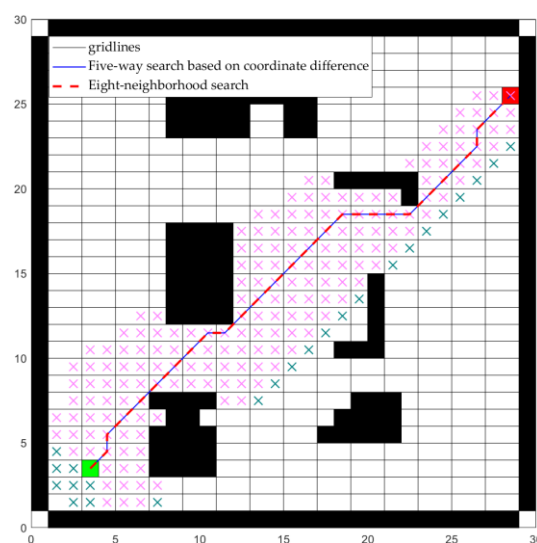
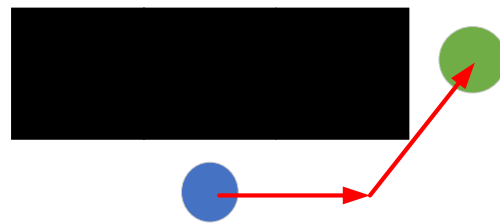


Figure 5. Comparison of five-way search and eight-neighborhood search based on coordinate differences.

Table 2. Eight-neighborhood and five-way search based on coordinate difference search results.

Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m
Five-way search based on coordinate difference	2.57	186	35.2843
Eight-neighborhood search	2.74	210	35.2843

From the table, the five-way search based on the number of coordinate changes reduces the number of search nodes and ensures a length, but it has the same situation with the eight-neighbor search, i.e., it will collide with the corner points of the obstacles, as shown in Figure 6. The blue circle is the current point, and the green circle is the next point to be searched.

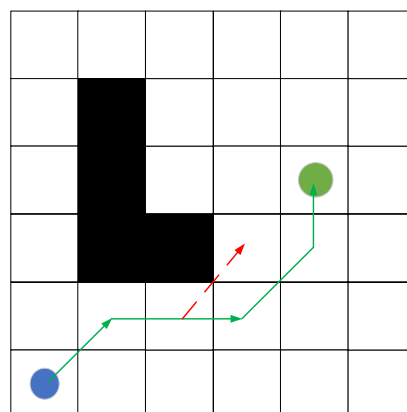
**Figure 6.** Schematic diagram of path and obstacle corner point collision.

Therefore, slash-free five-way search is proposed based on five-way search based on coordinate difference, and the selection rules are shown in Table 3.

Table 3. Sub-node selection rules for coordinate difference-based diagonal-free five-way searches.

Orientation of the Obstacle's Presence	Orientation of Child Node Removal
Above	1, 3
Right	1, 7
Bottom	5, 7
Left	3, 5

In this improved way, if the oblique expansion is still performed when there is no obstacle in the surrounding neighborhood, when there is an obstacle in the surrounding neighborhood, the algorithm filters out the oblique search and retains the other search directions, and the search is shown in Figure 7. The red dashed line is the path of the diagonal search, and the green line segment is the path planned based on the diagonal-free five-way search.

**Figure 7.** Coordinate difference-based diagonal-free five-way search maps.

Slash-free five-way search, five-way search, and eight-neighborhood search for comparison, and the comparison graph is shown in Figure 8, where the pink color is the nodes searched out of the eight-neighborhood multiple search, the blue line is the eight-neighborhood search path, the brown color is the five-way search path based on the difference of coordinates, and the red color is the no-slash five-way search path; comparative results are shown in Table 4.

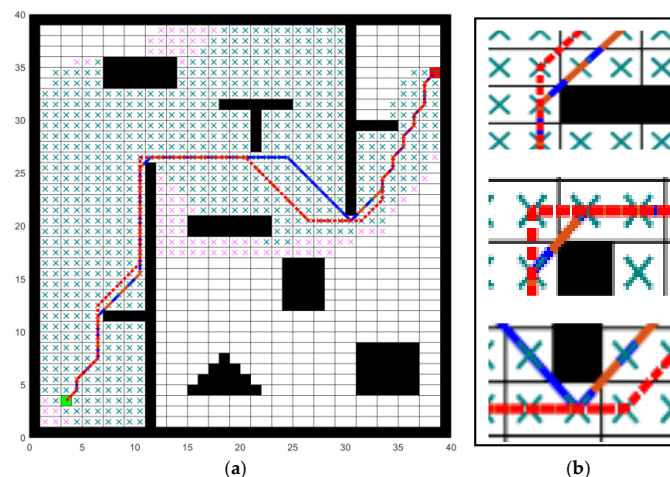


Figure 8. Three different neighborhood search graphs. They are: (a) Path comparison graph and (b) Detail graph.

Table 4. Search results for three different neighborhoods.

Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m	Amount of Hazardous Paths
Five-way search without slash	4.1921	704	66.2843	0
Five-way search	4.4327	704	65.1127	3
Eight-neighborhood search	6.2029	787	65.1127	4

As can be seen from the table, the diagonal-free five-way search is 1.7% longer than the optimal path in terms of path length because the search direction is changed when approaching the obstacle, but the algorithm time and the number of search nodes are reduced compared to the eight-neighborhood search. At the same time there are no more dangerous paths, so this paper will continue to study on the basis of diagonal-free five-way search. The flowchart of the coordinate difference-based diagonal-free, five-way search is shown in Figure 9:

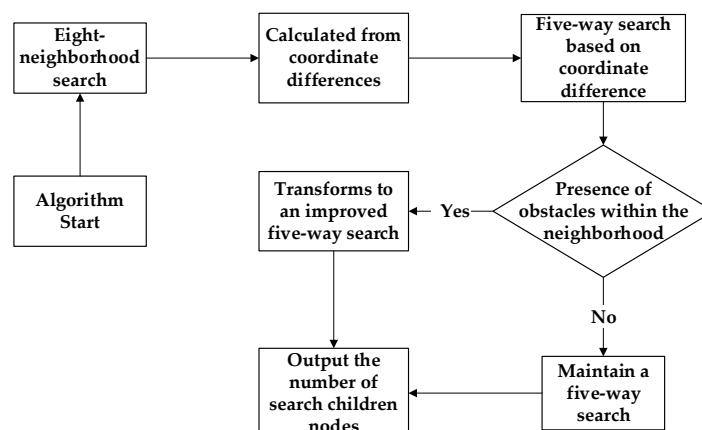


Figure 9. Flowchart of the node-expansion approach used in this paper.

2.3. Adaptive Dynamic Weighting Factors

The heuristic function provides speed and accuracy in the running of the algorithm [24]. Consequently, the amount of heuristic function will directly affect the final path-planning result. The actual results due to the weights of different heuristic functions are shown in Table 5:

Table 5. Path effects for different weights.

Weighting Situation	Path Effect
$h(n) = 0$	Equivalent to Dijkstra, it takes the longest time and searches the most nodes.
$g(n) = h(n)$	Essentially an A* algorithm with shorter time and shortest paths.
$g(n) \gg h(n)$	Longer time, speed proportional to $H(n)$ size, more extended nodes, optimal paths.
$g(n) \ll h(n)$	Similar to the greedy algorithm, the shortest time spent, fewest search nodes, longest paths.

According to the above table, to ensure the path optimization while speeding up the search, it is imperative to maintain equilibrium between the $h(n)$ and the $g(n)$. In this paper, it is hypothesized that at the beginning of the algorithm, the discrepancy between the $h(n)$ and the $g(n)$ is significant, and the number of search nodes should be reduced. As the algorithm approaches the target point, the discrepancy between the $h(n)$ and the $g(n)$ decreases gradually, and the way of searching the nodes should be changed from having a purposive search to a broad search.

Therefore, this paper argues that the amount of the $f(n)$ in the evaluation function should be dynamically controlled. For this reason, the expression of the improved evaluation function is shown in Equation (6):

$$f(n) = g(n) + f(x, y) * h(n) \quad (6)$$

The adaptive dynamic weight function ($f(x, y)$) is incorporated into the $f(n)$. The expression of the function is demonstrated in Equation (7):

$$f(x, y) = A \exp \left(-\frac{(x - \mu_x)^2}{2\sigma_x^2} - \frac{(y - \mu_y)^2}{2\sigma_y^2} \right) + k \quad (7)$$

where the coordinates of the current point are (x, y) ; μ_x and μ_y is the central point of the function, which is also the position of the maximum value of the function and, in this paper, represents the starting point; σ_x and σ_y is the standard deviation of the function in the x and y directions that determines how much the function expands; A is a constant, representing the amplitude of the function; and k is the magnitude of the translation on the y axis.

As shown in Figure 10, the shape and distribution of the whole radial basis function can be adjusted by changing σ_x and σ_y . If σ is too small, it will lead to a fast rate of change of the weights of the heuristic function so that the reduction of the number of search nodes is not obvious; on the other hand, if σ is too large, it cannot guarantee the relative optimality of the path.

Therefore, in this paper, after many experiments, it is concluded that σ in the range of 6–8 is the best, which can not only reduce the number of nodes but also ensure the relative path optimality.

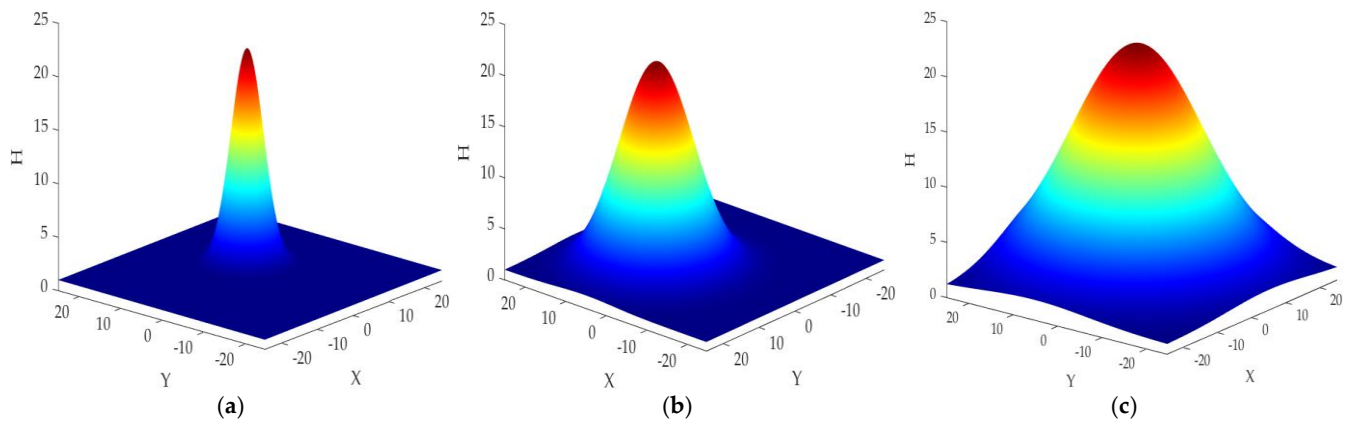


Figure 10. Three-dimensional plot of radial basis functions with different standard deviations. They are: (a) $A = 20, \sigma_x = 4, \sigma_y = 4$; (b) $A = 20, \sigma_x = 6, \sigma_y = 6$; (c) $A = 20, \sigma_x = 12, \sigma_y = 12$.

Set the amplitude as 20, the starting point coordinate as (5,5), $k = 1$, and the standard deviation as 8, and draw the three-dimensional contour plot and two-dimensional plane of the function, as shown in Figure 11.

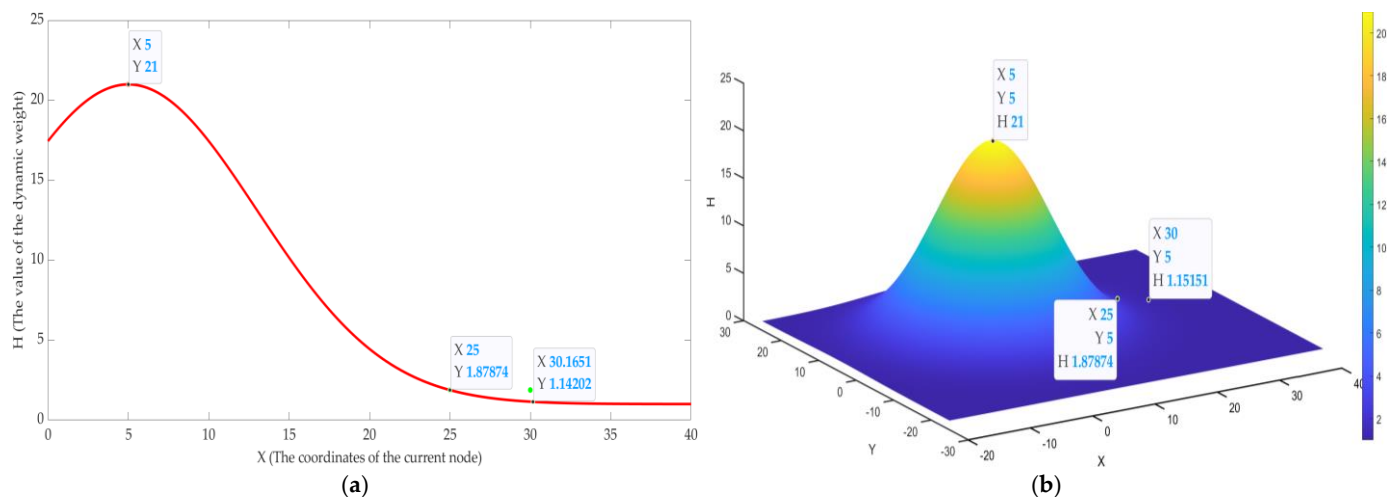


Figure 11. Two-dimensional plan and three-dimensional contour map. They are: (a) two-dimensional plan; and (b) three-dimensional contour map.

As shown in the figure, before $x = 25$, the value of the dynamic weight decreases rapidly. After $x = 25$, the value of the dynamic weight begins to decrease gently, and when $x = 30$, the value of the dynamic weight approaches 1. In the event of the node being in close proximity to the initial point, the dynamic weight undergoes an increase. Conversely, when the node moves away from the initial point, the dynamic weight undergoes a decrease, albeit a gradual one. This results in the search method becoming equivalent to the conventional A* algorithm when it attains a value of 1.

Since the magnitude of the value of this function converges infinitely to 1 and there is no strict minimum, in order for the equation to hold, a transformation is performed according to Equation (7), assuming that the calculation of the Euclidean distance from the current point to the initial point is conducted when the weights are equal to 1.1, as shown:

$$1.1 = 20 \cdot \exp\left(-\frac{(x-5)^2 + (y-5)^2}{2 \cdot 8^2}\right) + 1 \quad (8)$$

$$5.2983 = \frac{(x-5)^2 + (y-5)^2}{128} \quad (9)$$

$$(x-5)^2 + (y-5)^2 \approx 677.58 \quad (10)$$

Combined with the formula, the dynamic weight coverage is a circle with a radius of about 26. If the distance is greater than 26, the dynamic weights converge to 1 infinitely, and the algorithm is equivalent to the A* algorithm, thus realizing the dynamic change of weights. For the above case, it is hypothesized that the distance between the current point and the initial point is represented by the variable d , the actual results obtained by the heuristic function are shown in Table 6, and the dynamic weight change process is represented by colors on the raster map, as shown in Figure 12:

Table 6. The effect of the algorithm corresponding to the distance d .

Distance Between Current Point and Starting Point	Algorithmic Effect
$0 < d < 20$	Quickly search toward the target point, reducing the number of nodes traversed.
$20 \leq d \leq 26$	Slow decrease in search speed while increasing the number of nodes traversed.
$d > 26$	Slowest search speed, equivalent to traditional A* algorithm.

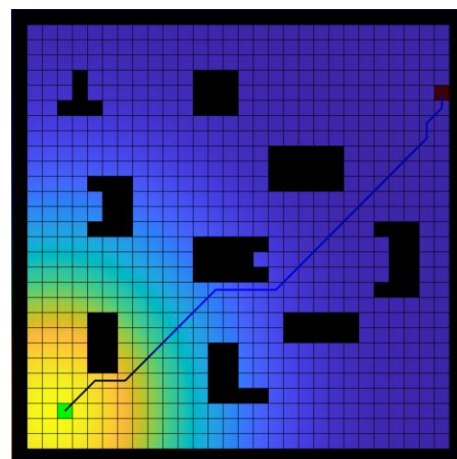


Figure 12. Dynamic weight change graph.

The purpose of this paper is to verify the effect of adding adaptive dynamic weights. This paper builds a 30×30 raster map with an obstacle scale of 0.25, and a comparison was made between the enhanced algorithm and the A* algorithm. The comparison graph is shown in Figure 13; the pink color represents the nodes searched by the two algorithms, the blue line segments are the paths planned by the A* algorithm, and the red line segments are the paths planned by adding adaptive weight, and the comparison results are shown in Table 7.

The A* algorithm with adaptive dynamic weights not only reduces the algorithm time but also reduces the number of nodes by about 40.3% and removes the hazardous paths. However, the length increases by about 2.2%, which is because the adopted adaptive dynamic weights continue to improve on the basis of the diagonal-free five-way search, and the algorithm automatically filters out the diagonal search when there are obstacles in the neighborhood.

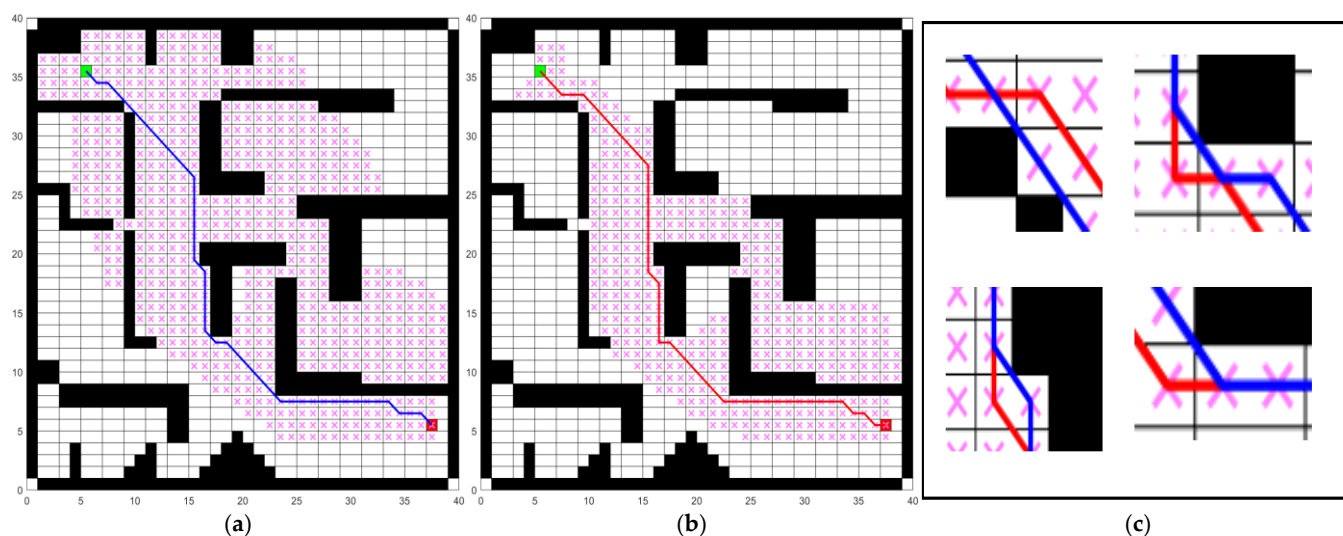


Figure 13. Comparison plot of A* algorithm with adaptive dynamic weighting. They are: (a) A*; (b) Adaptive dynamic weights; and (c) Path detail graphs.

Table 7. Comparison results between the A* algorithm and adaptive dynamic weighting.

Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m	Number of Hazardous Paths
A* algorithm	4.9142	691	51.4558	4
Adaptive dynamic weighting	2.5066	412	52.6274	0

2.4. Heuristic Reward Value

The A* algorithm using adaptive dynamic weight coefficients can dynamically change the weight coefficients, but if it encounters a long obstacle, the long obstacle blocks the direction of algorithm expansion, causing the algorithm to repeat visits to the open list, so it will appear to consume a lot of time, as shown in Figure 14.

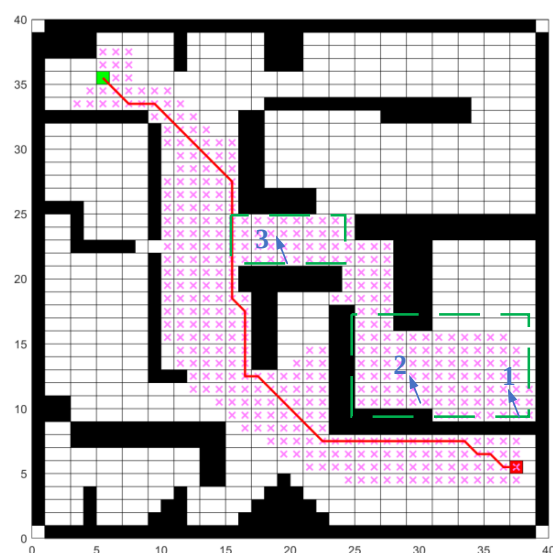


Figure 14. Plunging into a locally optimal node graph.

Nodes searched for using adaptive dynamic weights are marked in purple in the figure. In the green box it will take a lot of time; this is because when the algorithm expands to the labelled 1 grid, and then the next step in the expansion is found, the expansion of the nodes has been stored in the open table, and the expansion toward the target point

is blocked. It can therefore not be directly down the expansion. In order to ascertain the correct path, nodes will need to repeat the visit to the open list and select the least costly node from the open list until the visit to the current point is 3; then the algorithm exits the local optimal phenomenon.

This paper optimizes the cost function to let the target points be reverse diffused and cover the diffused reward values on the raster map, and the cost function adopts a heuristic approach to obtain these reward values so that the algorithm can escape from the phenomenon of local optimum. The flowchart of the steps of reward value expansion is shown in Figure 15.

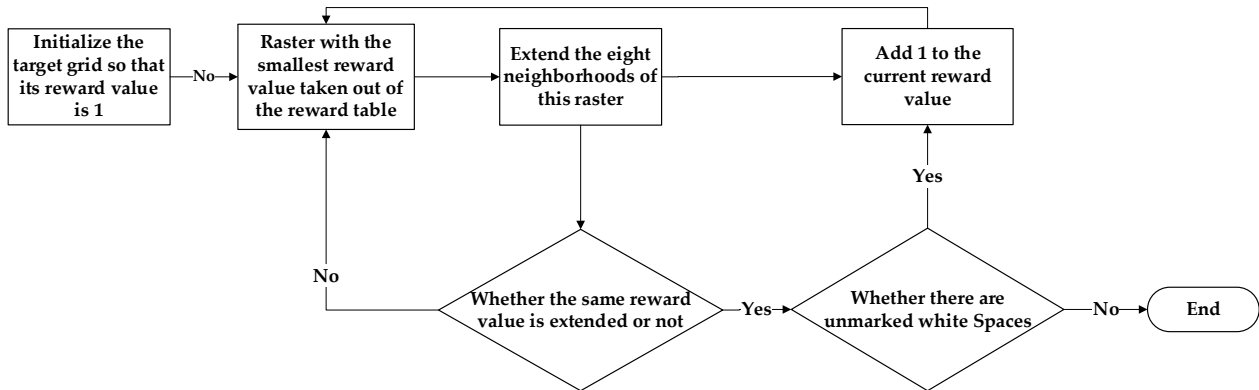


Figure 15. Reward value extension flowchart.

The cost function for increasing the value of the heuristic reward is shown in Equation (11):

$$f(n) = g(n) + f(x, y) * h(n) + t * value \quad (11)$$

In Equation (14), *value* is the reward value of the marker in the raster map, and *t* is a factor.

When the algorithm falls into the local optimum, assuming that A is the current node, the algorithm selects the smallest reward value around A, and then, through the guidance of the reward value, can escape from the local optimum. Such as A through the two expansions of the algorithm, the cost function selects the minimum cost of the grid, such as grid B, and through B continues to select the minimum cost of the grid, to the current grid to become C before you can leave the obstacle, as shown in Figure 16.

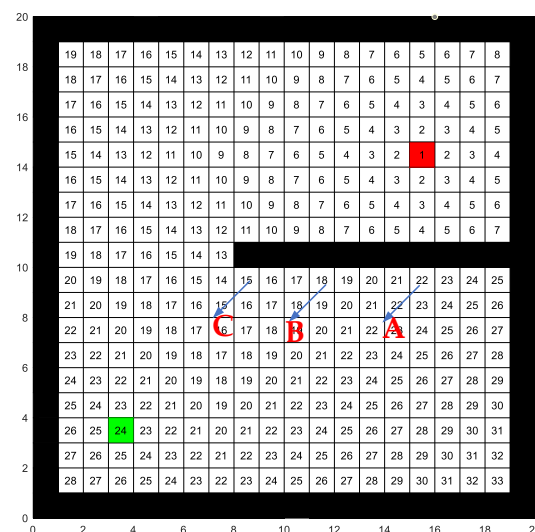


Figure 16. Reward value guided node escape graph.

A 40×40 raster map was created to compare the A* algorithm using adaptive dynamic weights with the A* algorithm adding heuristic reward values on top of it, as demonstrated in Figure 17; the comparison graph is presented, while the comparison results are outlined in Table 8. Where the pink color is the node searched by the algorithm, the blue line segment is the path planned by the A* algorithm using only adaptive weights, and the red line segment is the path planned by the A* algorithm incorporating adaptive weights and heuristic reward values.

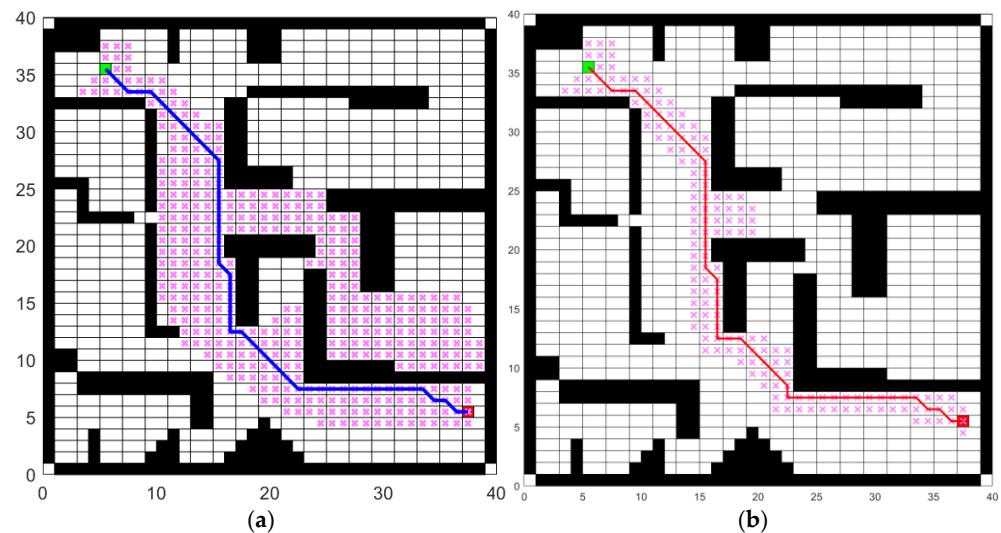


Figure 17. Comparison of the resultant graphs. They are: (a) Adaptive dynamic weights; and (b) Added heuristic reward values.

Table 8. Comparison results between the two.

Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m
Adaptive dynamic weighting	2.5066	412	52.6274
Increasing the value of heuristic rewards	2.4127	164	53.2132

The combination of heuristic reward values and adaptive dynamic weights has been shown to significantly reduce the number of search nodes; this approach can decrease the number of search nodes by 76.2% in comparison with the conventional A* algorithm. Furthermore, when contrasted with the number of nodes exhibited by the A* algorithm with solely adaptive dynamic weight coefficients, a reduction of 60.1% in search nodes can be observed, but the path length grows by 1.1%, which can be improved by the post-processing of the paths.

Simultaneous improvement of the A* algorithm using coordinate difference-based diagonal-free five-way search, adaptive dynamic weights, and heuristic reward values can significantly reduce the algorithm's time and the number of nodes and also avoid collision of paths with the corner points of the obstacles.

2.5. Secondary Optimization of Paths

The search angles of the improved A* algorithm are all multiples of 45 degrees, so there are many inflection points. The inflection points are mainly used to bypass obstacles; some of them are valid, but some of them are invalid redundant points, and these invalid inflection points not only increase the path length but also increase the turning angle. Therefore, a method of path quadratic optimization is proposed.

In this paper, we use the method of vector fork product to determine whether two line segments intersect or not, as shown in Figure 18. The formula for the two-dimensional vector fork product is shown below:

$$v_1 \times v_2 = x_1 \cdot y_2 - y_1 \cdot x_2 \quad (12)$$

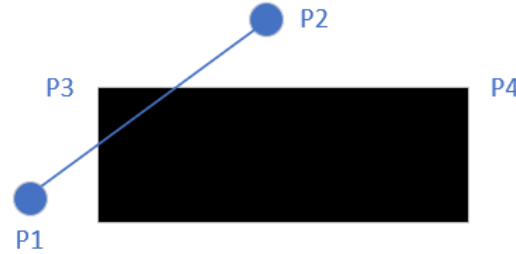


Figure 18. Schematic diagram of intersection with an obstacle.

$p1(x1, y1), p2(x2, y2), p3(x3, y3), p4(x4, y4)$ in the plane. $p1$ and $p2$ are connected to form the line $p1p2$, and $p3$ and $p4$ are connected to form the line $p3p4$. Equation (13) is used to express the position of $p3$ with respect to $p1p2$, Equation (14) is used to express the position of $p4$ with respect to $p1p2$, Equation (15) is used to express the position of $p1$ with respect to $p3p4$, and Equation (16) is used to express the position of $p2$ with respect to $p3p4$.

$$det1 = (x_2 - x_1) \cdot (y_3 - y_1) - (y_2 - y_1) \cdot (x_3 - x_1) \quad (13)$$

$$det2 = (x_2 - x_1) \cdot (y_4 - y_1) - (y_2 - y_1) \cdot (x_4 - x_1) \quad (14)$$

$$det3 = (x_4 - x_3) \cdot (y_1 - y_3) - (y_4 - y_3) \cdot (x_1 - x_3) \quad (15)$$

$$det4 = (x_4 - x_3) \cdot (y_2 - y_3) - (y_4 - y_3) \cdot (x_2 - x_3) \quad (16)$$

If $det1 \times det2 < 0$, it means that points $p3$ and $p4$ are on both sides of $p1$ and $p2$, and if $det3 \times det4 < 0$, it means that points $p1$ and $p2$ are on both sides of $p3$ and $p4$. When both conditions are satisfied, it means that the two line segments are intersecting.

The schematic diagram for detecting corner point collisions with obstacles is shown in Figure 19.

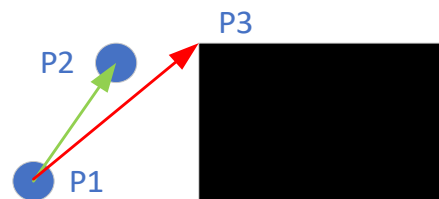


Figure 19. Schematic diagram of a corner point collision with an obstacle.

Define $vector1 = p2 - p1 = (x2 - x1, y2 - y1)$, $vector2 = p3 - p1 = (x3 - x1, y3 - y1)$, calculate the dot product of the two vectors and determine whether point $p3$ is on the line segment of $p1p2$. Based on the dot product, the dot product formula is shown below:

$$dot = (x_2 - x_1) \cdot (x_3 - x_1) + (y_2 - y_1) \cdot (y_3 - y_1) \quad (17)$$

The steps to implement the secondary optimization of paths are shown below:

(1) For the first optimization, the first point is the starting point. Three path nodes are selected each time, and it is determined whether the line connecting the two intersects with the obstacle. If it does not intersect, then $p2$ is called the redundant node. Remove the point, and then select the next node $p4$ of $p3$, always keeping three path nodes.

(2) If the intersection condition is satisfied, the intermediate node p2 must be retained. p3 is then treated as the first node on the path, the next node of p4 is selected, and the aforementioned steps are repeated until a connection is established with the target point. Then, the first optimization of the path is completed.

(3) In the second optimization, reverse for removal, the gap between the path and the obstacle after the first simplification is very small and dangerous. Therefore, the objective point is used as the starting point for the second optimization, and the above steps are repeated to carry out the removal in the reverse direction until it is connected to the starting point. Then, the second optimization is completed. After removing the redundant points two times, the optimized path can leave some gap with the obstacles.

The secondary optimization of the planned path is performed on the basis of Figure 16, the comparison diagram is shown in Figure 20, and the comparison results are shown in Table 9:

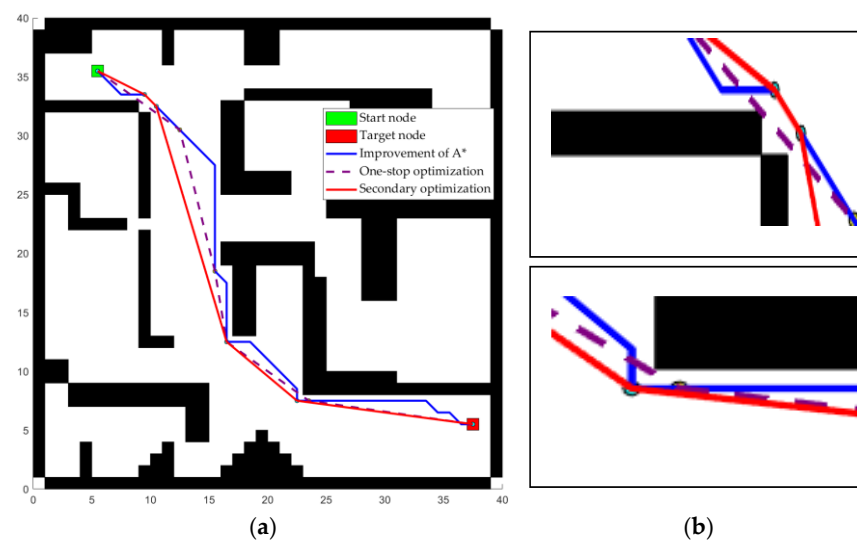


Figure 20. Path comparison charts. They are: (a) Three path comparison plots; and (b) Path detail diagram.

Table 9. Path optimization comparison.

Path Optimization	Number of Corners	Total Turning Angle/°	Path Length/m
No path optimization	13	585.0000	53.2132
Path primary optimization	4	117.4076	49.7989
Paths optimized twice	3	112.4418	49.7100

From the table, it can be seen that the path is optimized twice to reduce the number of corners, the number of rotation angles, and the path length, and also to widen the clearance from obstacles.

2.6. Bessel Curve Smoothing

The Bessel curve is derived from Bernstein polynomial evolution reasoning. The most important feature of the Bessel curve is that it can be accurately controlled by simple control points and can be smoothly connected. The n th order Bessel curve expression is shown below:

$$P(u) = \sum_{i=0}^n B_{i,n}(t)P_i \quad t \in [0, 1] \quad (18)$$

where P_i is the i th control point, $B_{i,n}$ is a polynomial formula where n represents the order of Bessel, and the number of control points and the order of Bessel satisfy: $i = n + 1$.

The third-order Bessel curve is shown in Figure 21, and its expression is shown below:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3 \quad (19)$$

where $P_0 P_1 P_2 P_3$ denotes the starting point, the two control points, and the end point, respectively, and t is a parameter.

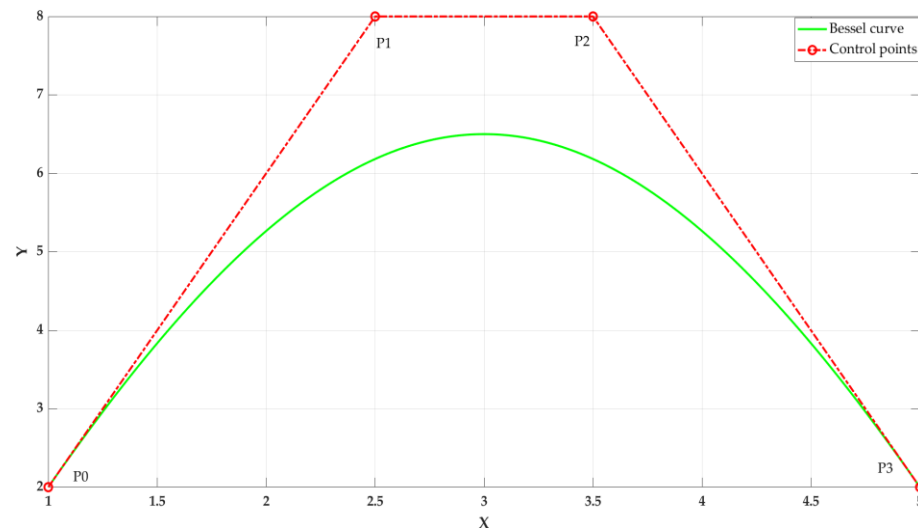


Figure 21. Third-order Bessel curve.

Since the distribution density of the nodes is different after the path is optimized twice, it may cause the smoothed path to collide with the obstacles inside the feature polygon. To solve this problem, we replan the simplified paths by adding auxiliary control points to make the smoothed paths fit the original paths more closely and not issue collisions with the internal obstacles, as shown in Figure 22. The blue circle is the path node, the red circle is the newly added auxiliary point, and the blue curve is the smoothed path.

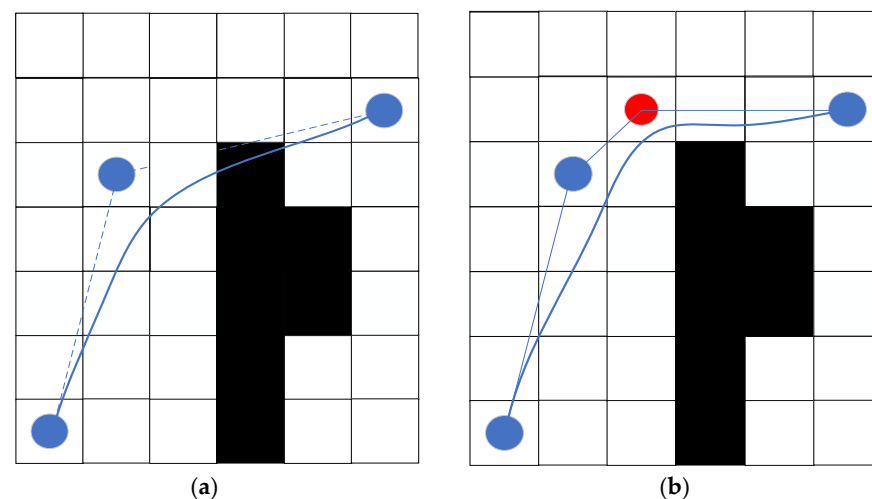


Figure 22. Comparison of Bessel curves. They are: (a) Second-order Bessel without added auxiliary points; and (b) Third-order Bessel with added auxiliary points.

On the basis of Figure 20, Bessel curve smoothing and three times B-spline smoothing are carried out, respectively, to compare the smoothing effect of the two, as shown in Figure 23, and the comparison results are shown in Table 10.

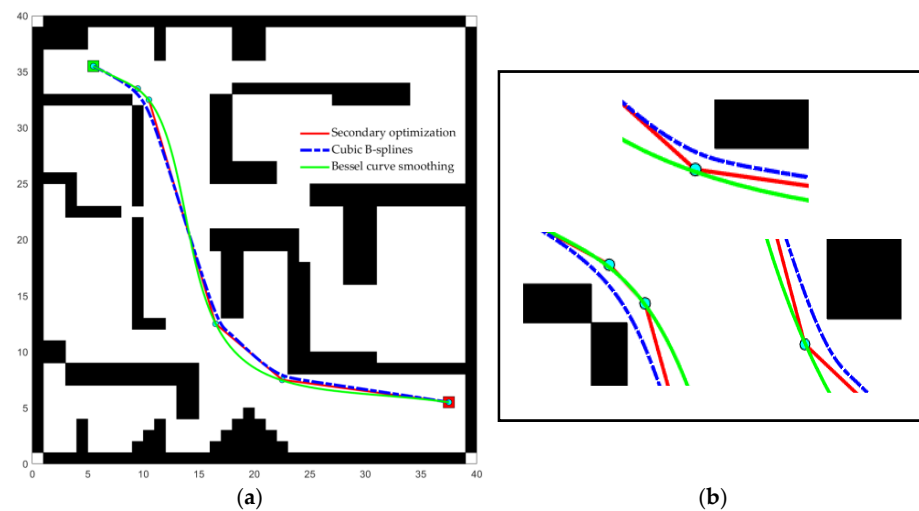


Figure 23. Smoothing effect comparison. They are: (a) Path comparison diagram; and (b) Path detail map.

Table 10. Path smoothing comparison.

Path Optimization	Number of Corners	Path Length/m	Change in Distance to Obstacles
Paths optimized twice	3	49.7100	-----
Bessel curve	0	50.0900	Increase
Cubic B-splines	0	49.0400	Decrease

Comparative analyses showed that the path after Bezier curve smoothing was slightly longer than the path after three B-spline smoothing. However, the distance between the B-spline smoothed path and the obstacle is significantly smaller than the spacing between the path and the obstacle after the second optimization process, but the Bessel curve increases the gap between the smoothed path and the obstacle compared to the path after the second optimization process, which improves the safety of the path. Meanwhile, Bezier curves are easy to understand and implement, and the shape of the curve can be directly controlled by adjusting the control points, so Bezier curves are chosen for smoothing in this study.

This concludes the improved A* algorithm of this paper, and the basic principle flowchart is shown in Figure 24.

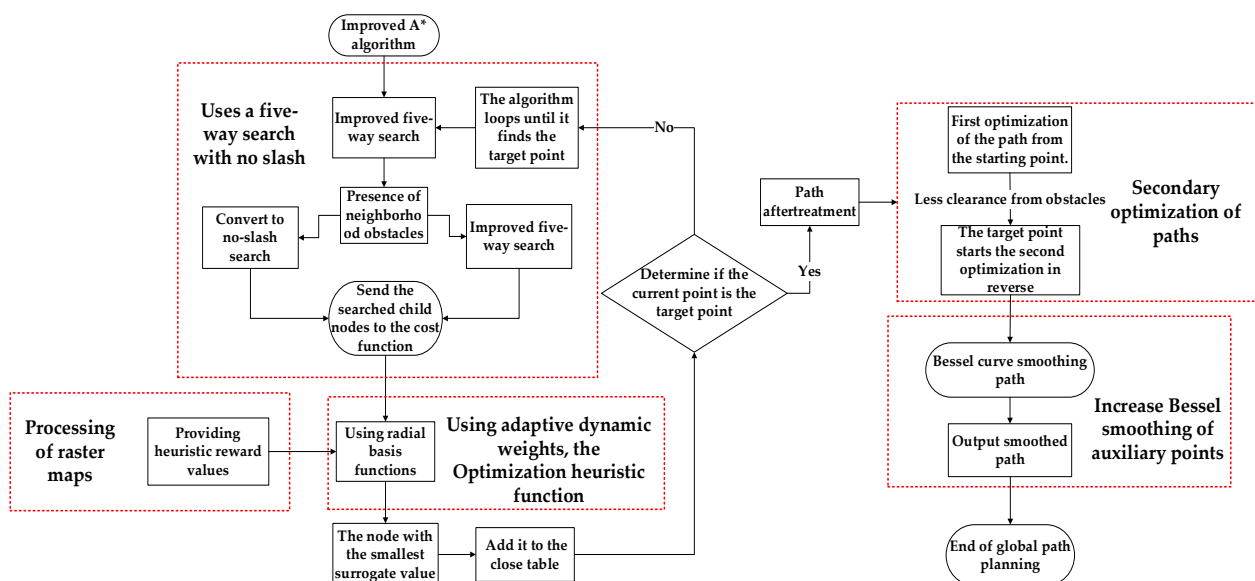


Figure 24. Flowchart of the improved A* algorithm in this paper.

3. Simulation Verification

In order to validate the enhanced algorithm presented in this paper, three sets of raster maps with different working conditions were constructed, and MATLAB R2022b was used to check the robustness and efficiency of the algorithm; the CPU was an i5-9300HF, and the graphics card was an NVIDIA GeForce GTX 1650.

Among the mainstream path-planning algorithms, the hybrid A* algorithm is optimized for motion constraints and continuous space problems and is able to efficiently plan paths under constraints; the D* streamlining algorithm is mainly applied to dynamic environments and is able to quickly adapt to changes in environments and adjust to changes in local obstacles. The main optimization goal of the improved algorithms in this paper is to improve the search speed and path quality of the algorithms in static environments by reducing redundant searches and using better heuristic functions, and dynamic constraints and the handling of dynamic obstacles are not taken into account. However, the RRT algorithm emphasizes the ability to explore unknown regions and search for target points quickly in larger and more complex environments and hence can be compared with this paper's algorithm in terms of search time. The Dijkstra algorithm and the A* algorithm can be used in a static environment to compare the number of nodes searched and the quality of planned paths with the algorithms in this paper.

Therefore, the RRT algorithm, Dijkstra's algorithm, and the A* algorithm are selected for comparison in this paper.

The green nodes are the nodes searched by the A* algorithm; the pink nodes are nodes searched by the Dijkstra algorithm more than A*; and the blue nodes are the nodes searched by the enhanced algorithm presented in this paper. The first two graphs of each group compare the number of nodes, and the last two graphs compare the algorithm paths.

3.1. Working Condition I

The simulation and validation environments use 30×30 , 40×40 , and 60×60 raster maps, respectively, and each group has the same location of the start and finish points, but the distribution of the obstacles is different.

3.1.1. 30×30 Raster Map

The improved algorithm was compared with other algorithms in a 30×30 raster map and the comparison graph is shown in Figure 25, and the comparison results are shown in Table 11.

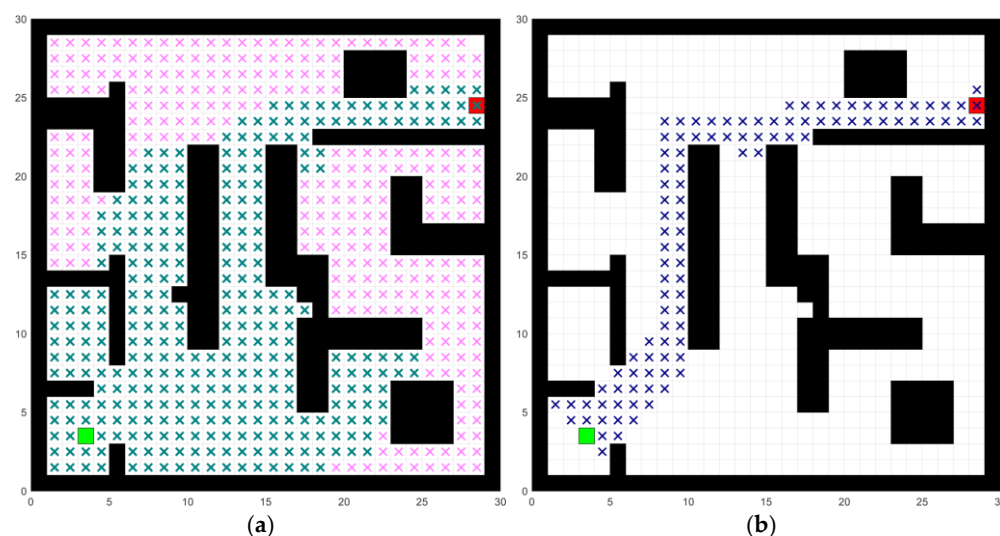


Figure 25. Cont.

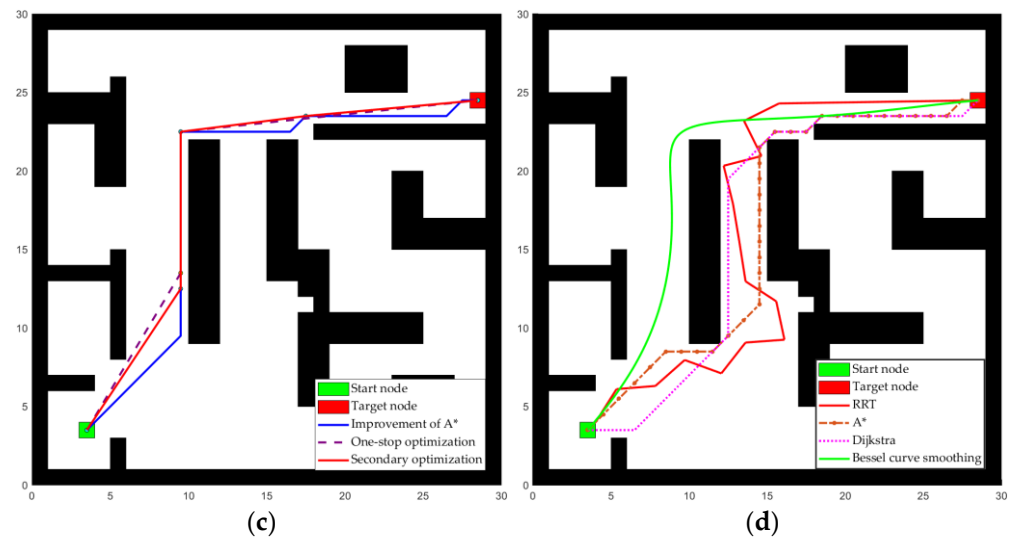


Figure 25. Simulation verification diagram. They are: (a) Number of nodes in A* vs. Dijkstra; (b) Number of nodes to improve A* in this paper; (c) Path optimization comparison; and (d) Path comparison of four algorithms.

Table 11. Comparison results of working condition I.

Map Size	Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m	Number of Hazardous Paths	Total Turning Angle/ $^{\circ}$
30×30	A*	2.3058	328	39.5563	3	405.0000
	Dijkstra	5.3000	616	39.5563	3	315.0000
	RRT	2.1012	—	47.5885	1	773.2819
	Secondary Optimization A*	1.4939	104	39.1446	0	118.4956
40×40	A*	6.0378	787	65.1127	4	900.0000
	Dijkstra	16.0523	1244	65.1127	4	585.0000
	RRT	4.5341	—	81.3228	2	1442.0000
	Secondary Optimization A*	2.5502	261	61.6190	0	171.5586
60×60	A*	12.7885	1336	85.4975	2	945.0000
	Dijkstra	18.7934	2821	85.4975	2	225.0000
	RRT	7.5721	—	114.1412	1	1046.8000
	Secondary Optimization A*	5.0533	318	80.7448	0	89.9469

3.1.2. 40×40 Raster Map

The improved algorithm was compared with other algorithms in a 40×40 raster map and the comparison graph is shown in Figure 26.

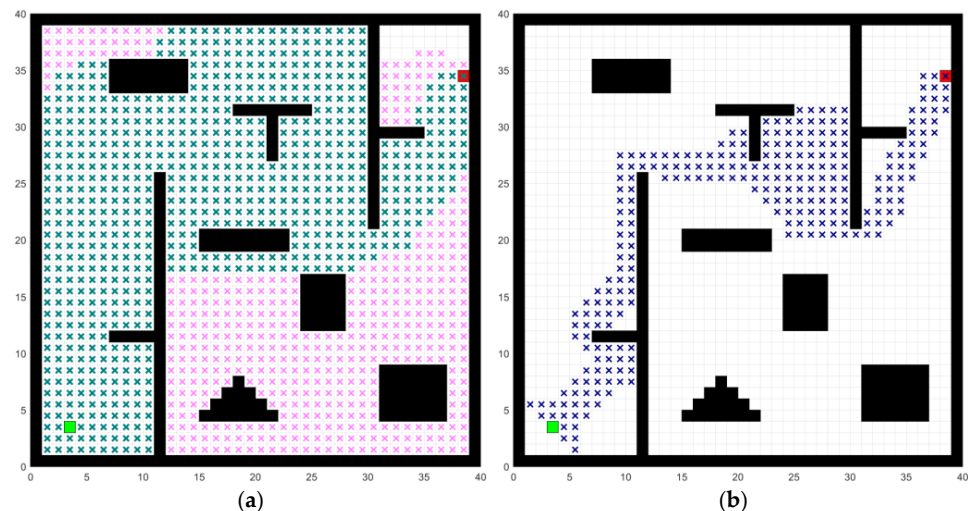


Figure 26. Cont.

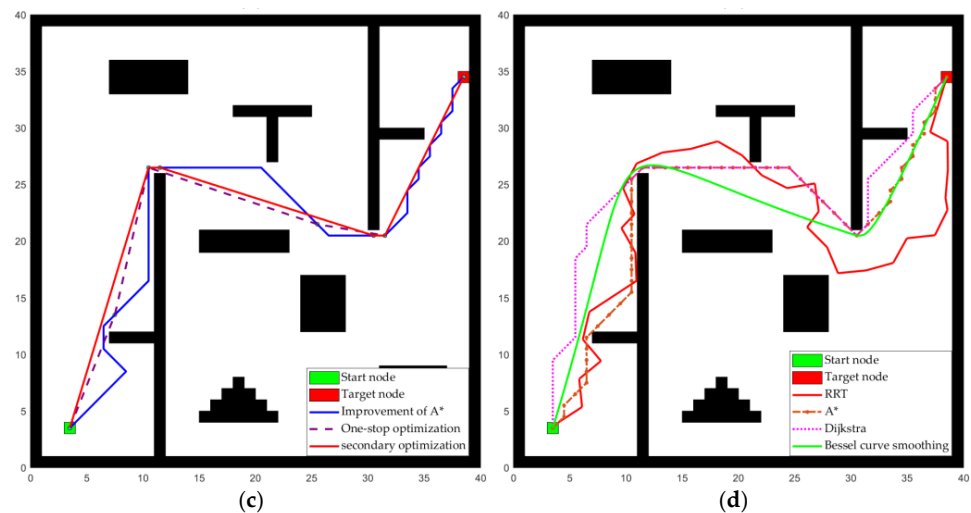


Figure 26. Simulation verification diagram. They are: (a) Number of nodes in A* vs. Dijkstra; (b) Number of nodes to improve A* in this paper; (c) Path optimization comparison; and (d) Path comparison of four algorithms.

3.1.3. 60 × 60 Raster Map

The improved algorithm was compared with other algorithms in a 60 × 60 raster map and the comparison graph is shown in Figure 27.

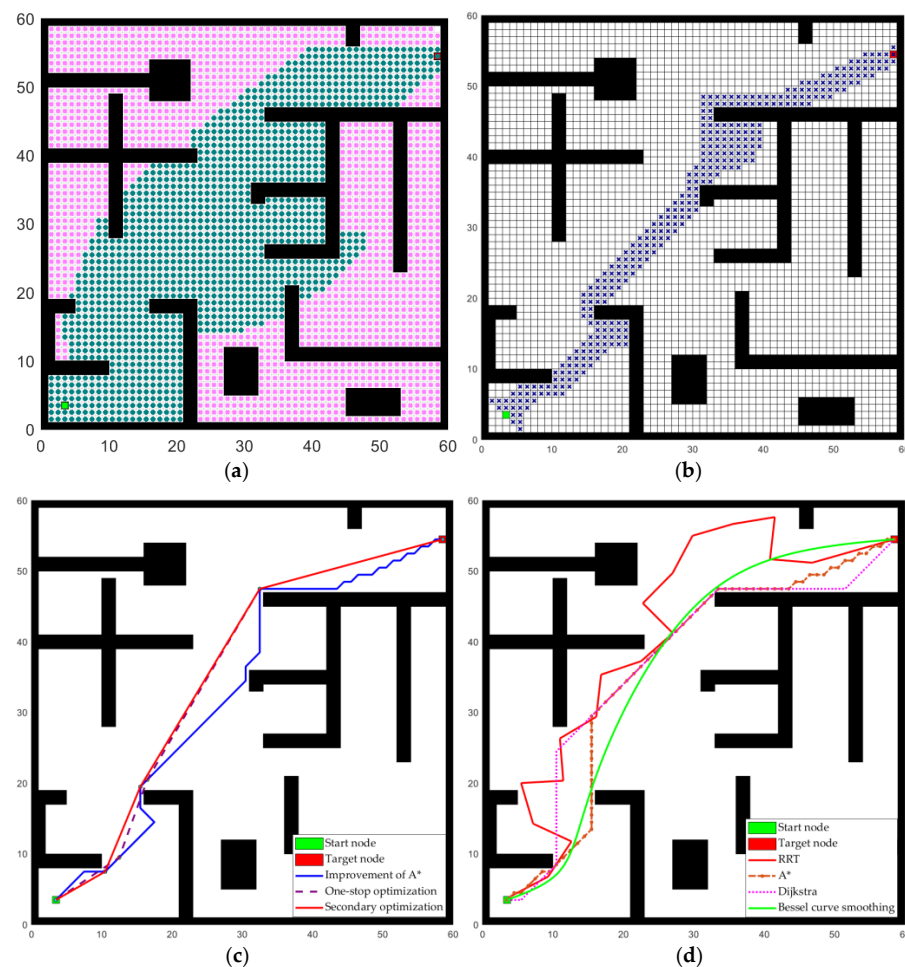


Figure 27. Simulation verification diagram. They are: (a) Number of nodes in A* vs. Dijkstra; (b) Number of nodes to improve A* in this paper; (c) Path optimization comparison; and (d) Path comparison of four algorithms.

3.1.4. Analysis of Results for Case I

Compared to the other three algorithms, the quadratic optimized A* algorithm reduces the algorithm time, path length, and plans paths without dangerous paths. Meanwhile, in 30×30 maps, search nodes can be reduced by 68.3% and 83.1%, and corners can be reduced by 70.7% and 62.4% compared to the A* and Dijkstra algorithms; in 40×40 maps, nodes can be reduced by 66.8% and 79%, and corners can be reduced by 80.9% and 79.0%; in 60×60 maps, nodes can be reduced by 76.2% and 88.7%, and corners can be reduced by 90.5% and 60.0%.

3.2. Working Condition II

The raster maps are sized using 30×30 vs. 50×50 , respectively, and the first map has an obstacle percentage of 0.37, and the second map has a narrow passageway, which continues to validate the robustness of the improved algorithm.

3.2.1. Raster Map with 0.37 Obstacle Share

The improved algorithm was compared with other algorithms on a map with an obstacle share of 0.37, and the comparison graph is shown in Figure 28, the comparison results are shown in Table 12.

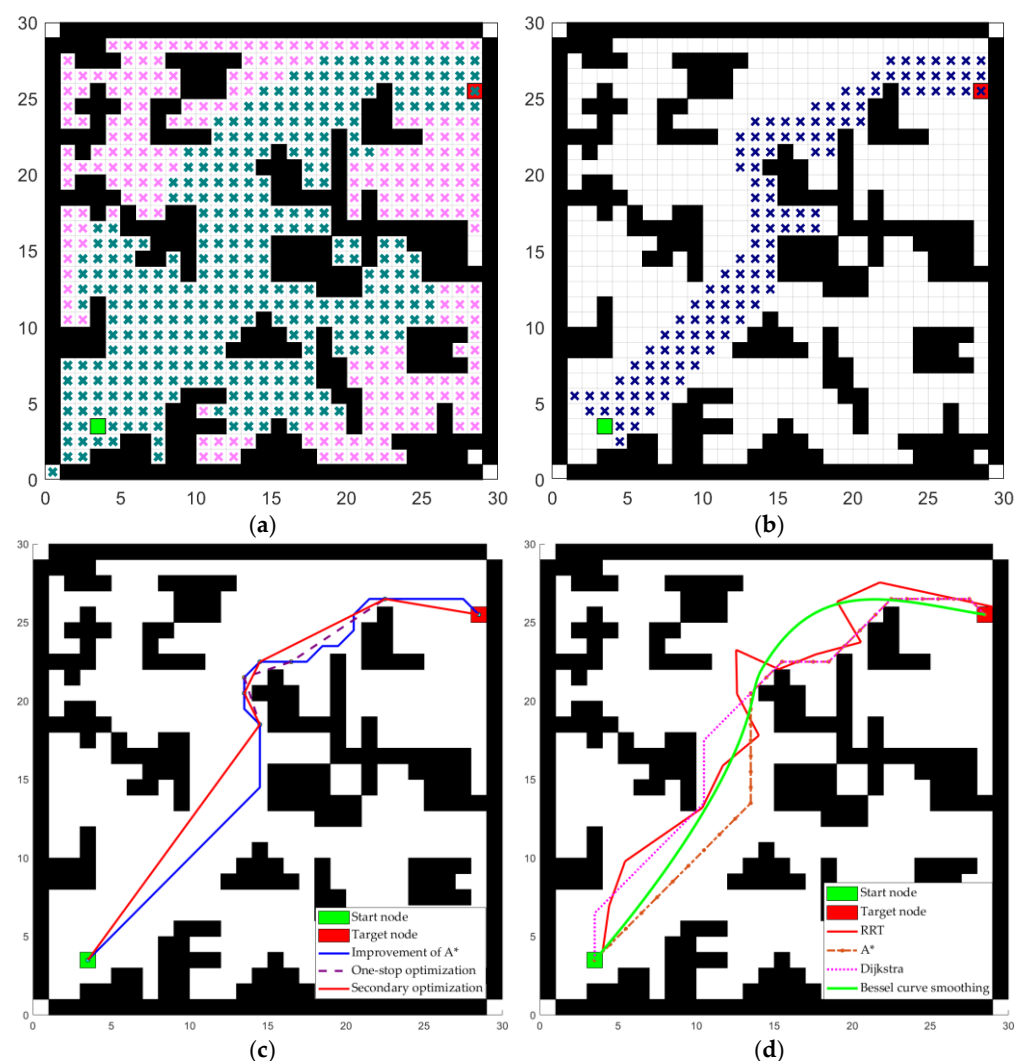


Figure 28. Simulation verification diagram. They are: (a) Number of nodes in A* vs. Dijkstra; (b) Number of nodes to improve A* in this paper; (c) Path optimization comparison; and (d) Path comparison of four algorithms.

Table 12. Comparison results of working condition II.

Map Type	Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m	Number of Hazardous Paths	Total Turning Angle/°
Percentage 0.37	A*	3.5030	347	39.0416	5	270.0000
	Dijkstra	5.0523	583	39.0416	6	315.0000
	RRT	4.6701	—	45.7576	2	612.5059
	Secondary Optimization A*	1.4906	124	37.7268	0	188.8463
Narrow passages	A*	7.1415	1064	66.2548	1	720.0000
	Dijkstra	16.0523	2133	66.2548	3	405.0000
	RRT	4.5341	—	90.1060	1	1177.6000
	Secondary Optimization A*	3.6318	276	62.4007	0	31.1273

3.2.2. Raster Map of Narrow Passages

The improved algorithm was compared with other algorithms on the map of narrow passages and the comparison graph is shown in Figure 29.

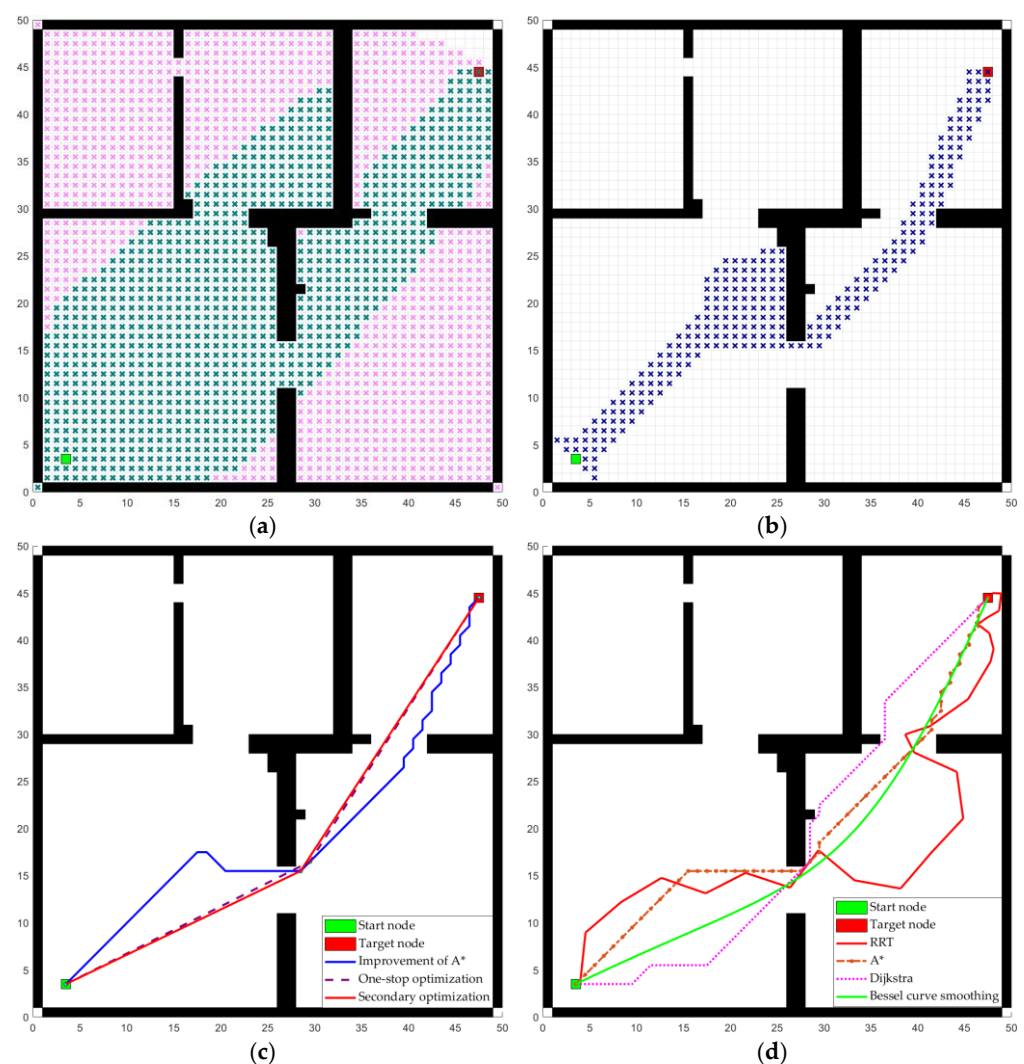


Figure 29. Simulation verification diagram. They are: (a) Number of nodes in A* vs. Dijkstra; (b) Number of nodes to improve A* in this paper; (c) Path optimization comparison; and (d) Path comparison of four algorithms.

3.2.3. Analysis of Results for Case II

In a map with an obstacle ratio of 0.37, search nodes can be reduced by 64.2% and 78.7%, and corners can be reduced by 30.1% and 40.1% compared to the A* and Dijkstra

algorithms; in the map with narrow passages, nodes can be reduced by 74.1% and 87.1%, and corners can be reduced by 95.7% and 92.3%.

3.3. Working Condition III

In order to verify the planning performance of the improved A* algorithm in large scenes, especially on the problems of large memory consumption, high computational complexity, and slow searching speed, a raster map with a size of 120×50 is constructed in this study to further verify the robustness and effectiveness of the algorithm in large scenes. Among them, the comparison graphs are shown in Figures 30–33, and the comparison results are shown in Table 13.

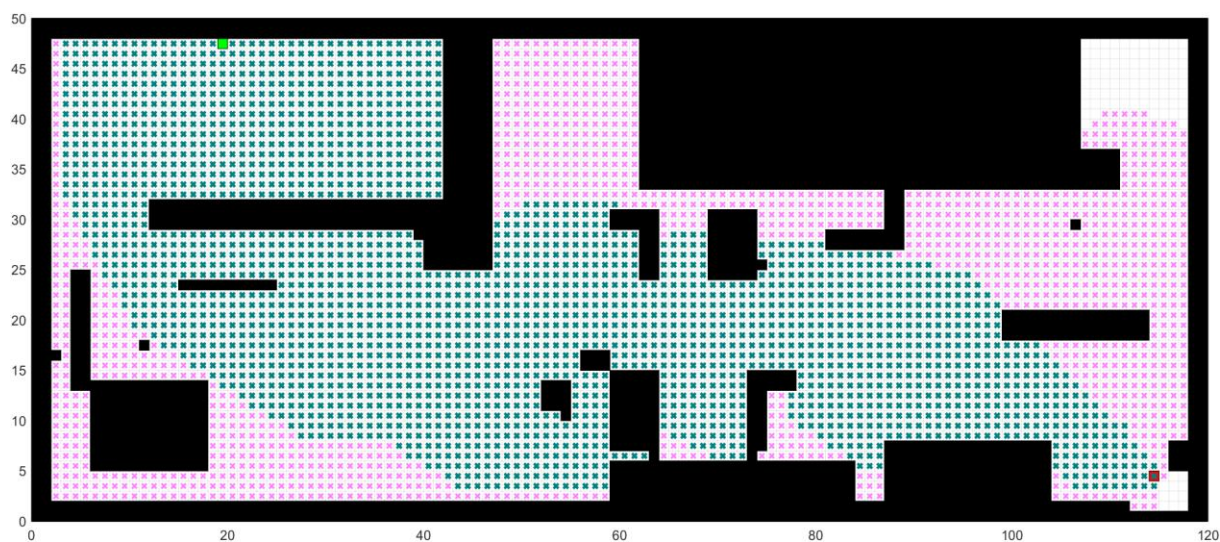


Figure 30. A* with the number of Dijkstra nodes.

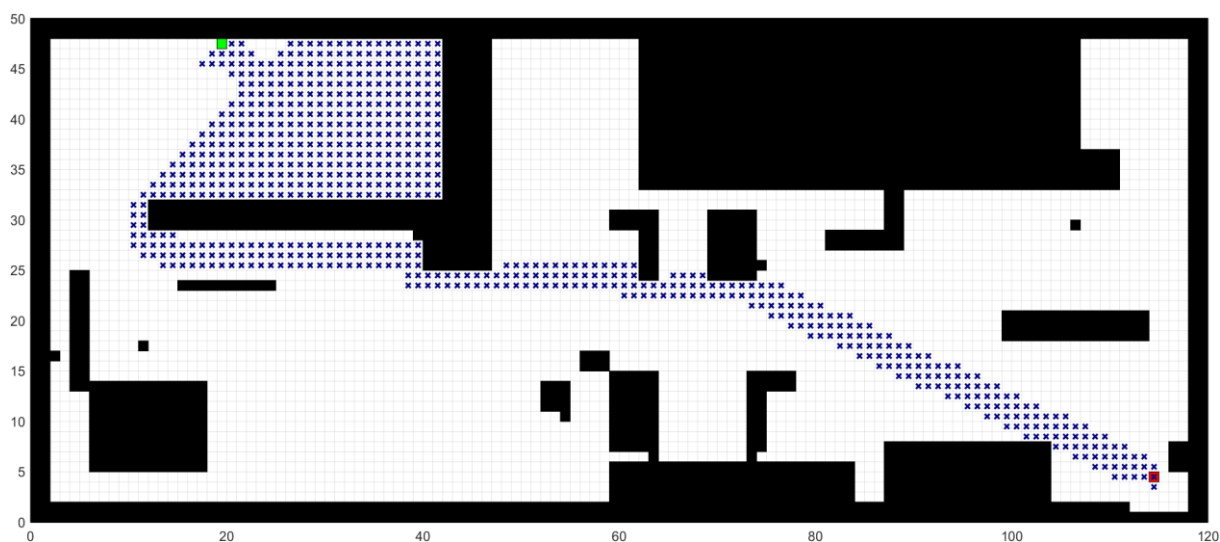


Figure 31. This paper improves the number of nodes in A*.

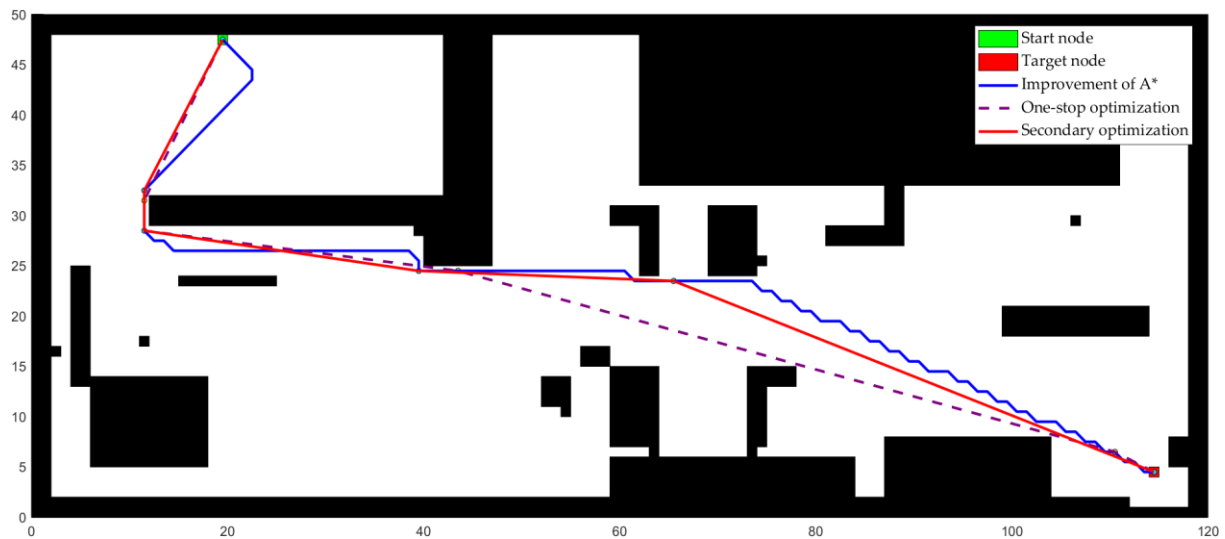


Figure 32. Path optimization comparison.

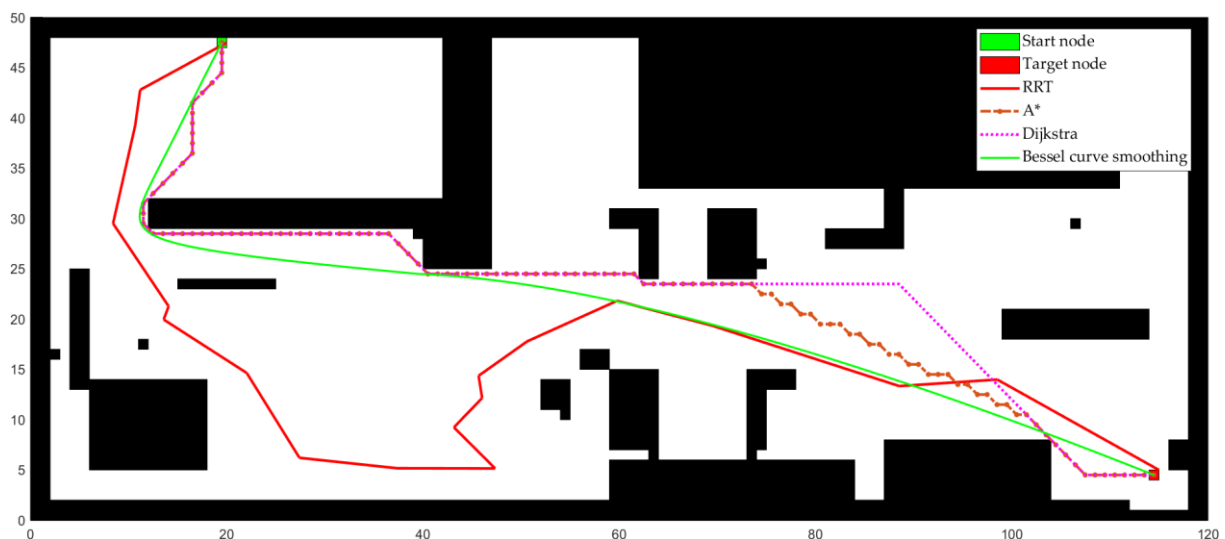


Figure 33. Comparison of four algorithmic paths.

Table 13. Comparison results of working condition III.

Map Size	Search Method	Algorithm Time/s	Number of Search Nodes	Path Length/m	Number of Hazardous Paths	Total Turning Angle/°
120 × 50	A*	12.8054	2497	134.6690	5	1710.0000
	Dijkstra	22.1499	3783	134.6690	5	540.0000
	RRT	18.5768	—	161.9879	0	774.0961
	Secondary Optimization A*	5.3214	742	127.8582	0	134.8613

Analysis of Results for Case III

Compared with the other three algorithms, the improved A* algorithm in this paper reduces search time and path length and eliminates dangerous paths in the planned route. At the same time, nodes can be reduced by 70.3% and total turning angle by 92.1% compared to A*, and nodes can be reduced by 80.4% and total turning angle by 75% compared to Dijkstra.

Collating the results of the three working conditions, it can be seen that the A* algorithm proposed in this paper not only minimizes the search time and path length but also reduces the number of search nodes by an average of 76.4% and reduces the total turning

angle by an average of 71.7%. In addition, the algorithm does not have any dangerous paths; this proves the robustness and effectiveness of the improved algorithm.

3.4. Compare the Search Time of the Algorithms

In order to verify the improvement in efficiency and complexity of the algorithm, the experimental maps are arranged in order from smallest to largest, and the algorithmic search times of the improved A* algorithm are compared with those of the RRT algorithm and the A* algorithm, as shown in Table 14.

Table 14. Comparative results.

Map Size	Search Time of the RRT Algorithm/s	Search Time of the A* Algorithm/s	Improving the Search Time of the A* Algorithm/s
30 × 30	2.1012	2.3058	1.4939
40 × 40	4.5341	6.0378	2.5502
50 × 50	4.5341	7.1415	3.6318
60 × 60	7.5721	12.7885	5.0533
120 × 50	18.5768	12.8054	5.3214

From the table, it can be seen that the improved A* algorithm reduces the search time of the algorithm compared to the RRT algorithm and the unimproved A* algorithm. However, the response time of the algorithm gradually increases as the size of the map increases. This is because when the size of the raster map increases, it leads to an increase in the search space. Each raster may need to be accessed and evaluated, and the number of nodes that need to be stored in the open list and the closed list increases, thus increasing the running time of the algorithm.

From the perspective of time complexity $O(b^d)$, b can denote the number of child nodes expanded from each parent node, and d can denote the distance from the starting point to the target, which may lead to the increase of both when the size of the map increases, thus causing an increase in the running time, especially when the map is of the same size; b will have a direct impact on the time complexity. The improved A* algorithm in this paper reduces the number of search nodes to a greater extent than the A* algorithm, which leads to a reduction in the number of newly expanded child nodes and therefore outperforms the A* algorithm in terms of search time.

4. Discussion

In this paper, based on the traditional A* algorithm, an A* algorithm is designed based on adaptive weights and heuristic reward values, which mainly includes the following:

(1) The diagonal-free five-way search based on coordinate difference is used to determine the search direction of the algorithm according to the position of the current point and the target point, and the redundant search direction is removed; and the algorithm filters out the diagonal search when there are obstacles in the search neighbourhood, to avoid the collision of the planned path with the corner points of the obstacles and to improve the path safety. Compared with the eight-neighborhood search, it not only reduces the algorithm search time but also reduces the number of search nodes by 10.5% and at the same time, avoids the appearance of dangerous paths.

(2) To solve the problem of A* searching a large number of nodes and taking a long time, we added an adaptive dynamic weighting factor based on a radial basis function to the heuristic function. This modification allows the algorithm to scale quickly, reducing the number of nodes traversed when moving away from the goal point, while allowing more nodes to be traversed when approaching the goal point. This modification saves the algorithm roughly half the search time and reduces the number of nodes searched by about 40.4%.

(3) In order to reduce the search time and the number of search nodes to a greater extent, a heuristic reward value from the target point is added to the cost function so that when the algorithm encounters a complex obstacle and a local optimum occurs, it can be guided out of the local optimum by the heuristic reward value. Experiments show that the algorithm with the heuristic reward value reduces the number of search nodes by about 60.2%.

(4) To address the issue of excessive corners and unnecessary nodes in the trajectory, a secondary optimization is performed to simplify the path by removing redundant points in both the forward and reverse directions and, at the same time, also increasing the distance between the path and obstacles, thereby improving the overall quality of the path. After the secondary optimization of the path, the number of corners is reduced by about 77.0%; the number of angles is reduced by about 80.8%, and the path length is reduced accordingly.

(5) Finally, to make the path smooth and continuous, additional auxiliary points are added to the simplified path, and Bessel smoothing is performed.

5. Conclusions

In this study, by improving the search direction, heuristic function, cost function, and path post-processing of the A* algorithm, the defects of the A* algorithm can be greatly improved, according to a series of experimental results that show that the improved A* algorithm not only reduces the search time and the path length, but also reduces the number of search nodes by about 76.6% on average, reduces the number of turning angles by about 71.6% on average, reduces the number of turning angles by about 69.3%, and at the same time, improves the path quality, which verifies the robustness and effectiveness of the algorithm.

There are also limitations to the algorithm:

(1) The algorithm lacks the ability to handle dynamic obstacles, ignores constraints related to obstacle avoidance, and does not integrate with local path-planning algorithms.

(2) Although the improved algorithm enlarges the distance between the path and obstacles and also performs Bessel smoothing on the path, it does not incorporate vehicle dynamics constraints in the global planning algorithm, which may result in the actual generated paths not being suitable for vehicle travelling.

(3) Although the improved A* algorithm performs significantly in a 2D environment, the limitation of the A* algorithm itself lies in a 3D environment [25], and future experiments can further enhance its performance in a 3D environment by introducing more complex heuristic functions, optimizing the search strategy, and incorporating techniques such as dynamic obstacle handling so as to overcome the problems of spatial complexity and computational burden.

For future research work, there is a need to incorporate vehicle dynamics constraints on the basis of global paths. In literature [26], dynamic constraints such as speed, acceleration limits, and minimum turning radius are considered to be able to generate trajectories that are more practically moving. Similarly, there is a need to further extend the distance to obstacles and achieve the ability to avoid both dynamic and static obligations with the help of the fusion of global and local planning algorithms to plan a safer and, at the same time, more convenient fusion path for the vehicle.

Author Contributions: Conceptualization, X.W. and G.L.; Methodology, X.W. and Z.B.; Software, X.W.; Validation, X.W. and G.L.; Formal analysis, X.W.; Investigation, X.W. and Z.B.; Resources, G.L. and X.W.; Data curation, X.W.; Writing—original draft preparation, X.W.; Writing—review and editing, X.W. and G.L.; Visualization, X.W. and G.L.; Supervision, X.W.; Project administration, G.L.; Funding acquisition, G.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Innovation Fund Project of Industry, University and Research of Chinese Universities (2024HT007) and the Key Research Project of Liaoning Provincial Department of Education (JYTZD2023081).

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Katona, K.; Neamah, H.A.; Korondi, P. Obstacle avoidance and path planning methods for autonomous navigation of mobile robot. *Sensors* **2024**, *24*, 3573. [\[CrossRef\]](#)
2. Wang, Z.; Li, G. Research on path planning algorithm of driverless ferry vehicles combining improved A* and DWA. *Sensors* **2024**, *24*, 4041. [\[CrossRef\]](#) [\[PubMed\]](#)
3. Li, J.; Liu, B.; Guo, D.; Gao, X.; Wang, P. An improved RRT path-planning algorithm based on vehicle lane-change trajectory data. *World Electr. Veh. J.* **2024**, *15*, 481. [\[CrossRef\]](#)
4. Li, L.; Jiang, L.; Tu, W.; Jiang, L.; He, R. Smooth and efficient path planning for car-like mobile robot using improved ant colony optimization in narrow and large-size scenes. *Fractal Fract.* **2024**, *8*, 157. [\[CrossRef\]](#)
5. Chatzisavvas, A.; Dossis, M.; Dasygenis, M. Optimizing mobile robot navigation based on A-Star algorithm for obstacle avoidance in smart agriculture. *Electronics* **2024**, *13*, 2057. [\[CrossRef\]](#)
6. Shang, E.; Dai, B.; Nie, Y.; Qi, Z.; Xiao, L.; Zhao, D. An improved A-Star based path planning algorithm for autonomous land vehicles. *Int. J. Adv. Robot. Syst.* **2020**, *17*, 1729881420962263. [\[CrossRef\]](#)
7. Tang, G.; Tang, C.; Claramunt, C.; Hu, X.; Zhou, P. Geometric A-Star algorithm: An improved A-Star algorithm for AGV path planning in a port environment. *IEEE Access* **2021**, *9*, 1. [\[CrossRef\]](#)
8. Xiang, D.; Lin, H.; Ouyang, J.; Huang, D. Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot. *Sci. Rep.* **2022**, *12*, 13273. [\[CrossRef\]](#)
9. Zhang, J.; Wu, J.; Shen, X.; Li, Y. Autonomous land vehicle path planning algorithm based on improved heuristic function of A-Star. *Int. J. Adv. Robot. Syst.* **2021**, *18*, 10. [\[CrossRef\]](#)
10. Li, X.; Hu, X.; Wang, Z.; Du, Z. Path planning based on combination of improved A-STAR algorithm and DWA algorithm. In Proceedings of the 2020 2nd International Conference on Artificial Intelligence and Advanced Manufacture (AIAM), Manchester, UK, 15–17 October 2020; pp. 99–103. [\[CrossRef\]](#)
11. Liu, L.; Wang, B.; Xu, H. Research on Path-Planning Algorithm Integrating Optimization A-Star Algorithm and Artificial Potential Field Method. *Electronics* **2022**, *11*, 3660. [\[CrossRef\]](#)
12. Zhang, H.; Tao, Y.; Zhu, W. Global path planning of unmanned surface vehicle based on improved A-Star algorithm. *Sensors* **2023**, *23*, 6647. [\[CrossRef\]](#)
13. Tang, X.; Zhu, Y.; Jiang, X. Improved A-star algorithm for robot path planning in static environment. *J. Phys. Conf. Ser.* **2021**, *1792*, 012067. [\[CrossRef\]](#)
14. Yu, M.; Luo, Q.; Wang, H.; Lai, Y. Electric Logistics Vehicle Path Planning Based on the Fusion of the Improved A-Star Algorithm and Dynamic Window Approach. *World Electr. Veh. J.* **2023**, *14*, 213. [\[CrossRef\]](#)
15. Guo, Z. Efficient path planning method based on A-STAR. *J. Phys. Conf. Ser.* **2023**, *2649*, 012026. [\[CrossRef\]](#)
16. Li, P.; Hao, L.; Zhao, Y.; Lu, J. Robot obstacle avoidance optimization by A* and DWA fusion algorithm. *PLoS ONE* **2024**, *19*, e0302026. [\[CrossRef\]](#) [\[PubMed\]](#)
17. Zhang, C.; Yang, X.; Zhou, R.; Guo, Z. A path planning method based on improved A* and fuzzy control DWA of underground mine vehicles. *Appl. Sci.* **2024**, *14*, 3103. [\[CrossRef\]](#)
18. Wang, X.; Zhang, H.; Liu, S.; Wang, J.; Wang, Y.; Shangguan, D. Path planning of scenic spots based on improved A* algorithm. *Sci. Rep.* **2022**, *12*, 1320. [\[CrossRef\]](#)
19. Ou, Y.; Fan, Y.; Zhang, X.; Lin, Y.; Yang, W. Improved A* path planning method based on the grid map. *Sensors* **2022**, *22*, 6198. [\[CrossRef\]](#)
20. Zhi, R.; Dong, Z.; Shuo, T. Reinforcement learning based improved 3D A* algorithm for online trajectory planning. *Syst. Eng. Electron.* **2023**, *45*, 193–201.
21. Kvitko, D.; Rybin, V.; Bayazitov, O. Chaotic Path-Planning Algorithm Based on Courbage–Nekorkin Artificial Neuron Model. *Mathematics* **2024**, *12*, 892. [\[CrossRef\]](#)
22. Ju, C.; Luo, Q.; Yan, X. Path planning using an improved a-star algorithm. In Proceedings of the 2020 11th International Conference on Prognostics and System Health Management (PHM-2020 Jinan), Jinan, China, 23–25 October 2020; pp. 23–26. [\[CrossRef\]](#)

23. Gao, J.; Xu, X.; Zhang, X.; Xu, S.; Pu, Q. Path Planning of Mobile Robot Based on Improved A* Algorithm. In Proceedings of the 2022 5th World Conference on Mechanical Engineering and Intelligent Manufacturing (WCMEIM 2022), Ma'anshan, China, 18–20 November 2022; pp. 1098–1102.
24. Sang, W.; Yue, Y.; Zhai, K.; Lin, M. Research on AGV Path Planning Integrating an Improved A* Algorithm and DWA Algorithm. *Appl. Sci.* **2024**, *14*, 7551. [[CrossRef](#)]
25. Xu, Y. The comparison of advanced algorithms for pathfinding robots. In Proceedings of the 2024 2nd International Conference on Computer Science and Mechatronics, Shanghai, China, 26–28 January 2024; Volume 3194. [[CrossRef](#)]
26. Rösmann, C.; Hoffmann, F.; Bertram, T. Kinodynamic trajectory optimization and control for car-like robots. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, Vancouver, BC, Canada, 24–28 September 2017; pp. 5681–5686. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.